

SLAM - Ex 5

Iris Kaplan & William Ayoub

June 20, 2026

This document contains the answers to VAN Exercise 5. The code for this exercise is in [this link](#). The final version used for this submission corresponds to commit .

1 Track Reprojection Error

In this section we evaluate one feature track using the camera poses estimated in Exercise 3. The global coordinate system was chosen as the coordinate system of the first left camera. The relative PnP transformations from Exercise 3 were composed into global extrinsic camera matrices $T_{0 \rightarrow i}$, where i is the frame index. These matrices were saved by `section_3_6` in `code/ex3.py`, line 304.

We loaded the tracking database constructed in Exercise 4 and randomly selected a track of length 15. For every frame in this track, we used the corresponding global camera matrix to construct a `gtsam.StereoCamera`.

Let the extrinsic camera matrix of a frame be $[R \mid t]$, where $R \in \mathbb{R}^{3 \times 3}$ and $t \in \mathbb{R}^3$. In our convention, this matrix maps points from the global coordinate system to the camera coordinate system:

$$x_{\text{cam}} = Rx_{\text{world}} + t.$$

However, `gtsam.StereoCamera` expects the pose in the opposite direction, from camera coordinates to global coordinates. Since $R^{-1} = R^T$, we get

$$x_{\text{world}} = R^{-1}(x_{\text{cam}} - t) = R^T x_{\text{cam}} - R^T t.$$

Therefore, the pose passed to GTSAM is

$$[R^T \mid -R^T t].$$

Using the `StereoCamera` of the last frame in the selected track, we triangulated a 3D point with `backproject`. Since the camera pose is given in the camera-to-global convention, the triangulated point is represented in global coordinates. We then projected this point into all frames of the track using `project`, and compared the projections with the measured stereo locations from the tracking database.

For frame i , let the measured and projected stereo points be

$$z_i = \begin{bmatrix} u_{L,i} \\ u_{R,i} \\ v_i \end{bmatrix}, \quad \hat{z}_i = \begin{bmatrix} \hat{u}_{L,i} \\ \hat{u}_{R,i} \\ \hat{v}_i \end{bmatrix}.$$

The stereo reprojection error plotted in Figure 1 is

$$\|z_i - \hat{z}_i\|_2.$$

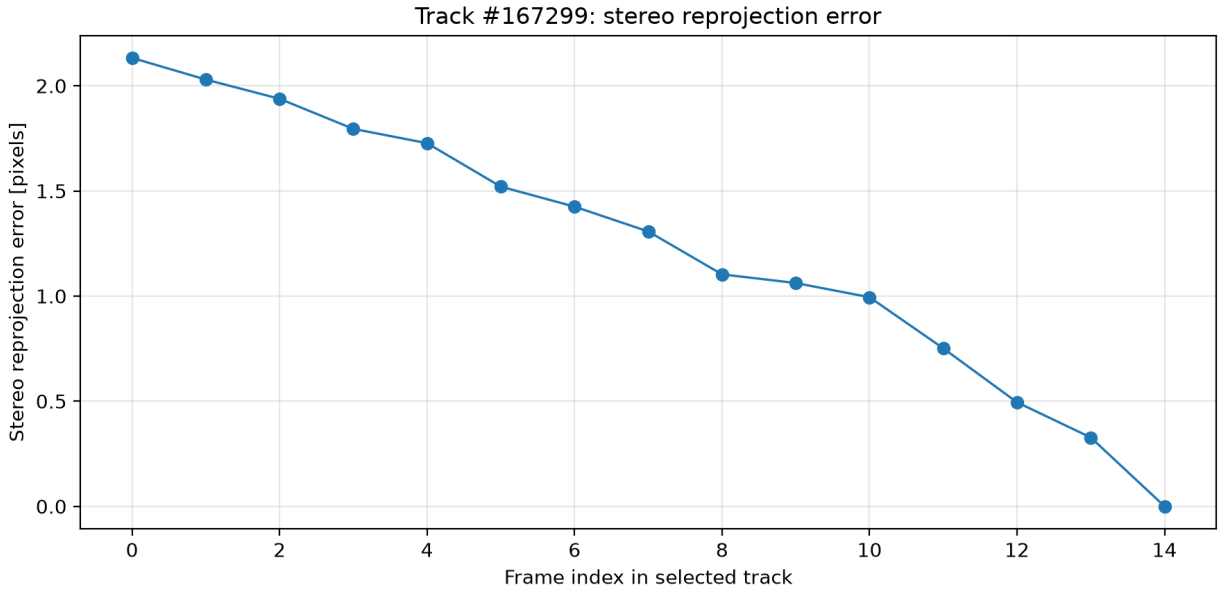


Figure 1: Stereo reprojection error over the selected track. The 3D point was triangulated from the last frame, so the error at that frame is approximately zero. The figure was generated by `question5_1` in `code/ex5.py`, line 65, using `plot_reprojection_errors_q5_1` in `code/slam/visualization/ex5_plots.py`, lines 21–50.

We also created one `gtsam.GenericStereoFactor3D` for each frame projection. We used an isotropic pixel noise model for the stereo coordinates (u_L, u_R, v) , with standard deviation $\sigma = 1$ pixel. Thus, the covariance matrix was

$$\Sigma = \sigma^2 I_3 = I_3.$$

Denoting the residual by $r_i = z_i - \hat{z}_i$, the stereo factor error is

$$E_i = \frac{1}{2} r_i^T \Sigma^{-1} r_i.$$

Since $\Sigma = I_3$, this becomes

$$E_i = \frac{1}{2} \|r_i\|_2^2.$$

Thus, the factor error is one half of the squared reprojection error. This relation is visible in Figures 1 and 2, which have the same qualitative shape.

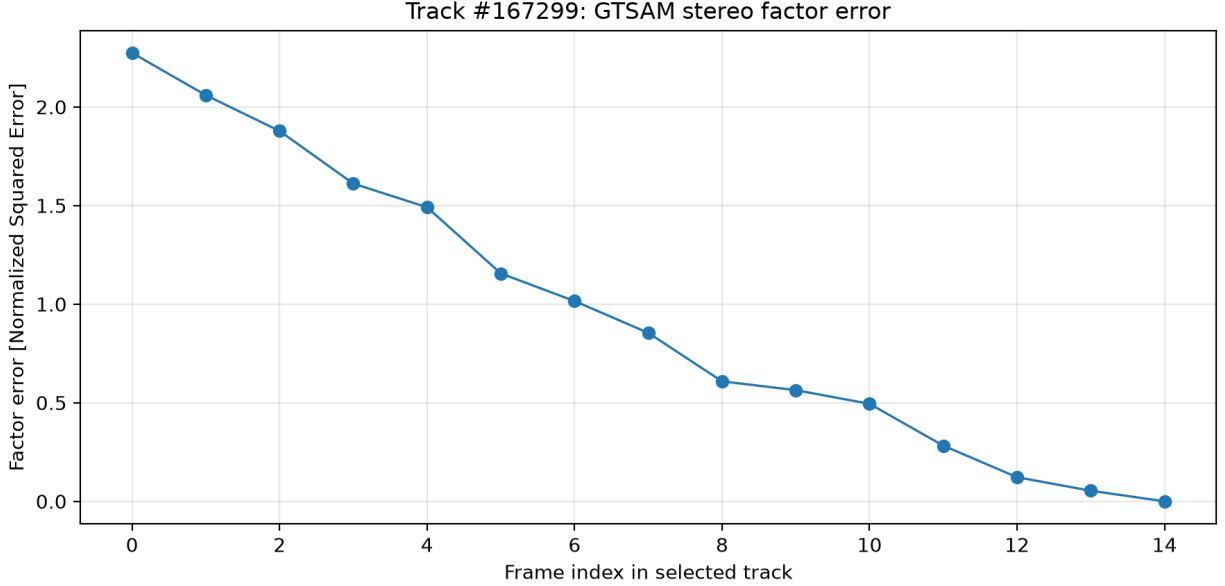


Figure 2: GTSAM stereo factor error over the selected track. The figure was generated by `question5_1` in `code/ex5.py`, line 65, using `plot_factor_errors_q5_1` in `code/slam/visualization/ex5_plots.py`, lines 53–82.

2 Transformation in a Common Coordinate System

Let cameras A and B have extrinsic matrices $[R_A \mid t_A]$ and $[R_B \mid t_B]$, respectively, with both extrinsics defined with respect to the same global coordinate system. Thus, for a global point x_g , its coordinates in camera A are

$$x_A = R_A x_g + t_A,$$

and its coordinates in camera B are

$$x_B = R_B x_g + t_B.$$

We want to express the transformation from the coordinate system of camera A to the coordinate system of camera B . First, we invert the transformation of camera A :

$$x_g = R_A^T(x_A - t_A).$$

Substituting this expression into the equation for camera B gives

$$x_B = R_B R_A^T(x_A - t_A) + t_B.$$

Rearranging terms, we obtain

$$x_B = R_B R_A^T x_A + (t_B - R_B R_A^T t_A).$$

Therefore, the transformation (and extrinsic matrix) from camera A coordinates to camera B coordinates is

$$T_{A \rightarrow B} = [R_B R_A^T \mid t_B - R_B R_A^T t_A].$$

3 Bundle Adjustment Window

For simplicity, we selected keyframes at a fixed interval of 10 frames, resulting in local windows of 11 consecutive stereo pairs.

The first bundle window spans frames 0–10. It contains 11 camera poses, 1810 landmarks, and 7710 factors, including one prior factor anchoring the first camera pose. Table 1 summarizes the factor-graph error before and after optimization.

Table 1: Optimization statistics for the first bundle window. The average factor error is the total graph error divided by the total number of factors. Values are printed by `_print_bundle_summary()` in `code/ex5.py`, lines 97–109.

Quantity	Value
Initial total graph error	8433.007
Final total graph error	1551.122
Number of factors	7710
Initial average factor error	1.094
Final average factor error	0.201

The optimization reduces the total graph error by approximately a factor of 5.4.

We next inspected the projection factor with the largest initial error. This factor connects camera pose $c = 3$ with landmark $q = 861$.

Table 2: Largest initial-error stereo projection factor before and after bundle adjustment.

Quantity	Initial values	Optimized values
Factor error	244.491	1.468
Left-image distance	15.412 px	1.713 px
Right-image distance	17.395 px	1.663 px

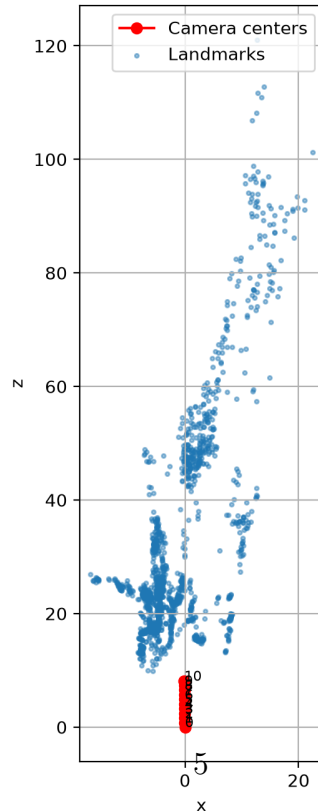
For both the initial and optimized estimates, we constructed a `StereoCamera` using the corresponding pose of c and projected the corresponding landmark q . Figure 3 compares the measured feature position with the projected position in the left and right images.

Figures 4 and 5 show the optimized scene for the first bundle window. Camera poses and landmarks are expressed relative to frame 0, which is the first frame of this specific window.



Figure 3: Stereo reprojection diagnostic for the factor with the largest initial error. The red cross denotes the stereo measurement and the green circle denotes the projected landmark. The top row uses the initial values and the bottom row uses the optimized values. The figure was generated by `question5_3` in `code/ex5.py`, line 191, using `plot_largest_factor_diagnostic` in `code/slam/visualization/ex5_plots.py`, lines 117–200.

Optimized local bundle: top-down view



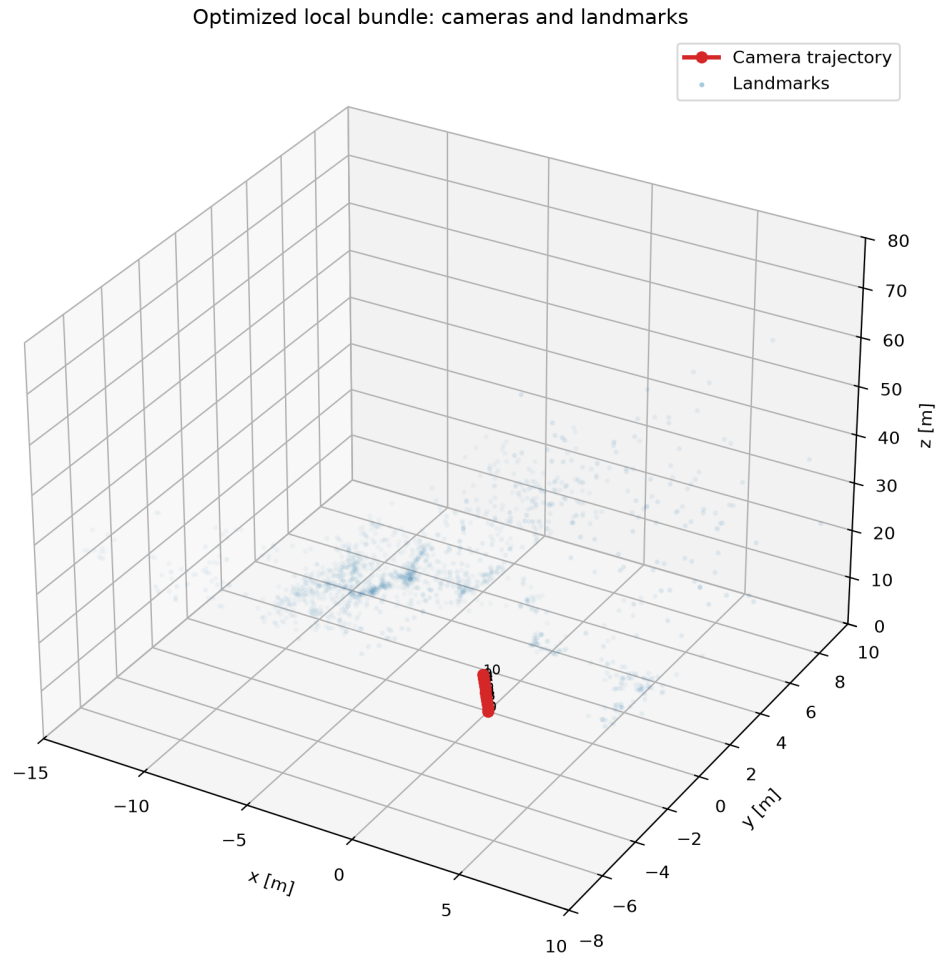


Figure 5: Three-dimensional view of the optimized first bundle window, including the camera trajectory and reconstructed landmarks. Axes are measured in meters. The figure was generated by `question5_3` in `code/ex5.py`, line 203, using `plot_bundle_scene_3d` in `code/slam/visualization/trajectory.py`, lines 52–99.

4 Sliding Bundle Windows

We chose all keyframes along the trajectory at a fixed interval of 10 frames and solved all resulting bundle windows independently. Each window uses the coordinate system of its first frame. The initial values for poses and landmarks were transformed from global coordinates into the first-frame-local coordinate system using the `relative_extrinsic` function in `code/slam/geometry/transforms.py`, line 112. A total of 330 bundle windows were solved.

4.1 Last Bundle Window Diagnostics

The last bundle window spans frames 3290–3299. After optimization, the exact position of the first frame of this window is $[-2.838 \cdot 10^{-14}, -7.081 \cdot 10^{-15}, -7.306 \cdot 10^{-15}]^T$, which is essentially $[0, 0, 0]^T$ up to machine precision. The anchoring factor’s final error is exactly 0.0 — this is expected because the prior factor strongly constrains the first pose to its initial value (which is defined as the identity in the local coordinate system of the bundle window), and the optimizer can satisfy this constraint completely with zero cost.

4.2 Global Pose Composition

From each solved bundle window we extracted the relative pose between adjacent keyframes using `start_pose.between(end_pose)` in `code/slam/ba/gtsam_utils.py`, lines 128–168. We then composed these relative poses into absolute global poses starting from frame 0:

$$T_{\text{global}}^{(k+1)} = T_{\text{global}}^{(k)} \circ T_{\text{relative}}^{(k \rightarrow k+1)}.$$

This composition accumulates drift over the full 3300-frame sequence because each local bundle window is solved independently without loop closure constraints.

4.3 Top-Down View with Ground Truth

Figure 6 shows the top-down view of all bundle-adjusted keyframes and landmarks, overlaid with the ground-truth keyframe positions from the KITTI dataset.

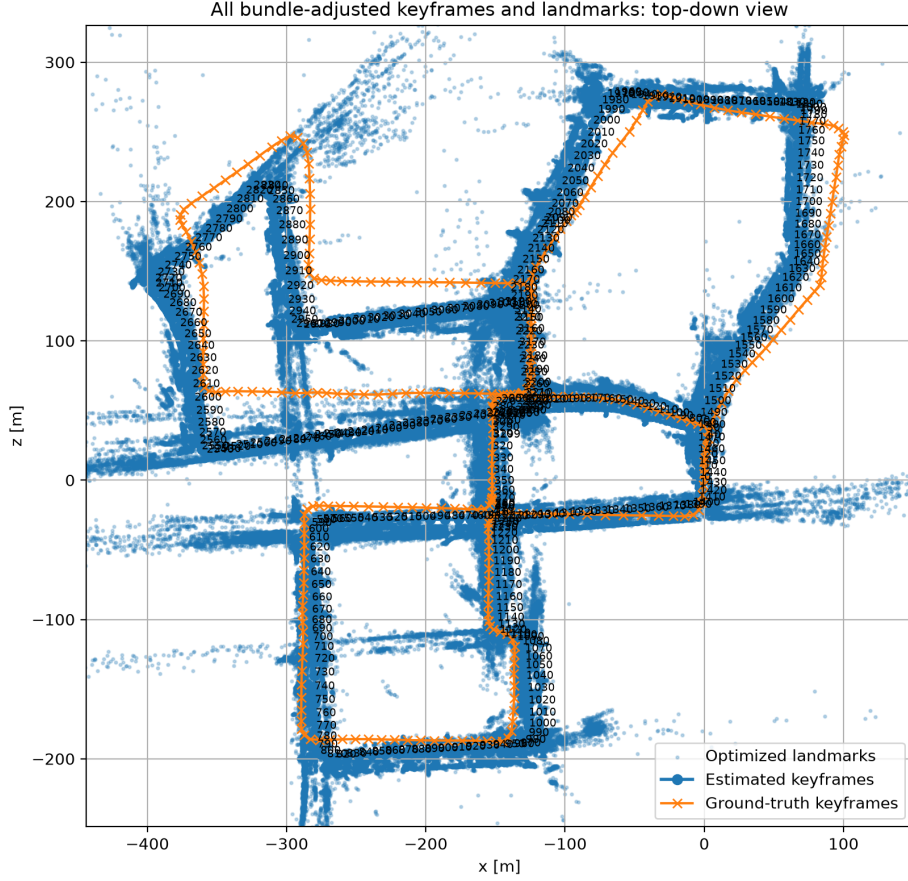


Figure 6: Top-down view of all bundle-adjusted keyframes (blue) and ground-truth keyframe positions (orange). Optimized landmarks are shown as light blue scatter points. The figure was generated by `question5_4` in `code/ex5.py`, line 277, using `plot_keyframes_and_landmarks_top_down` in `code/slam/visualization/ex5_plots.py`, lines 203–261.

4.4 Keyframe Localization Error

Figure 7 presents the keyframe localization error over time. The error is defined as the Euclidean distance between the estimated and ground-truth positions of each keyframe:

$$e_k = \|\hat{p}_k - p_k^{\text{GT}}\|_2.$$

The mean keyframe localization error is 22.159 m. The error grows and oscillates over the sequence, reflecting drift accumulation in the composed relative poses. The periodic drops correspond to segments where the vehicle revisits previously traversed areas (the KITTI 00 sequence contains several loops), temporarily reducing the apparent position error. However, since we perform only local bundle adjustment without explicit loop closure, the accumulated drift is not corrected globally, though 22 meters of drift over a multi-kilometer trajectory demonstrates the strong local accuracy of the bundle adjustment.

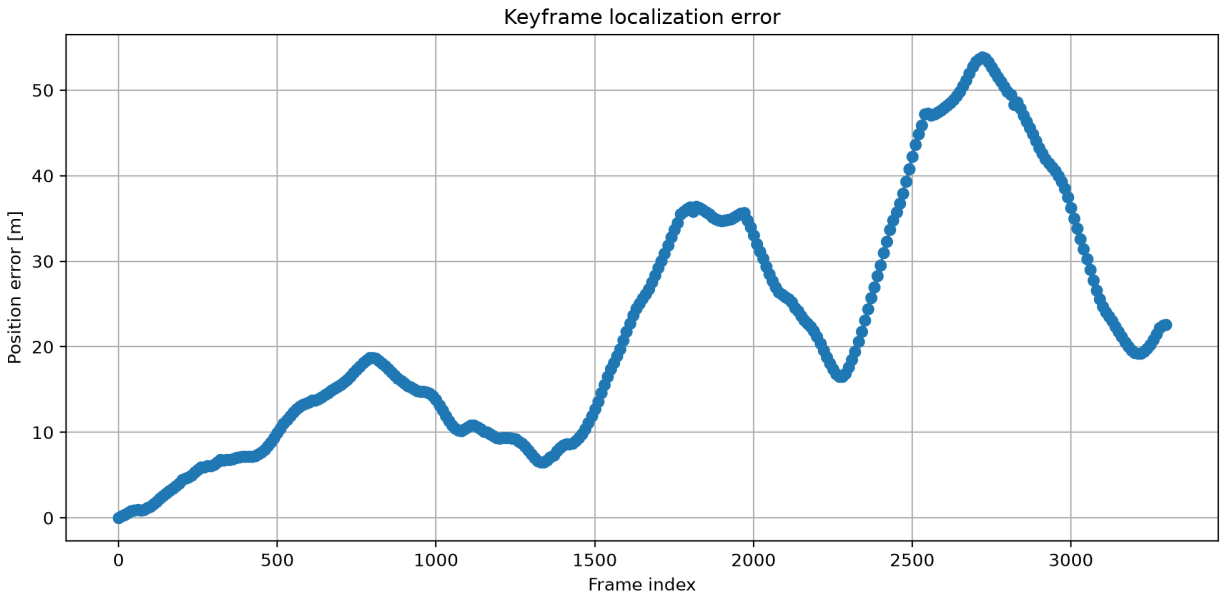


Figure 7: Keyframe localization error (Euclidean distance between estimated and ground-truth positions) over the frame index. The figure was generated by `question5_4` in `code/ex5.py`, line 285, using `plot_keyframe_localization_error` in `code/slam/visualization/ex5_plots.py`, lines 264–292.